

Documentación de diseño

Por: Aythami Reyes y Carlos Perdomo

Curso: 1 ºA DAW

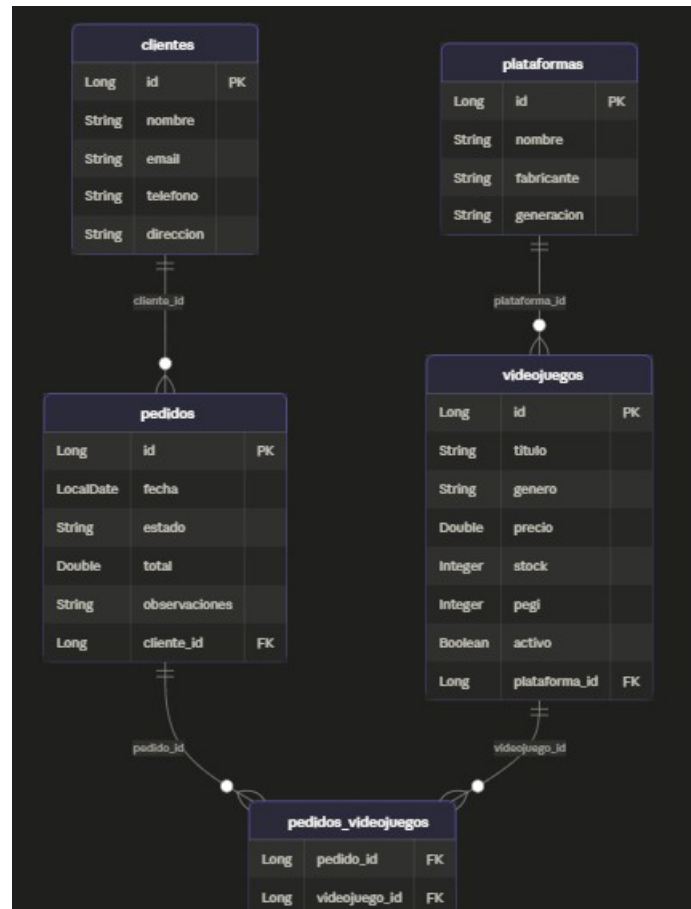


Indice

Documentacion de diseño.....	1
1. Diagrama ER con entidades, relaciones, PKs y Fks.....	3
3. Lista de endpoints con método, ruta y descripción.....	5
3.1 Plataformas.....	5
3.2 Videojuegos.....	5
3.3 Clientes.....	6
3.4 Pedidos.....	7
4. Capturas o vídeo de las tablas generadas en la BD.....	8
4. Decisiones técnicas justificadas.....	17
4.1 Base de datos: H2 en memoria.....	17
4.2 Arquitectura por capas: Controller → Service → Repository.....	17
4.3 Optional y ResponseEntity.....	17
4.4 @JsonIgnore para evitar referencias circulares.....	17
4.5 @ManyToMany con @JoinTable explícita.....	17
4.6 Métodos derivados y @Query JPQL.....	17
4.7 Seguridad con Spring Security Basic Auth.....	18
4.8 Validación con @Valid y manejo global de excepciones.....	18
4.9 Sort dinámico en búsquedas.....	18
5. Capturas de las tablas en la base de datos.....	19
6. Instrucciones para arrancar el proyecto.....	21

1. Diagrama ER con entidades, relaciones, PKs y Fks

Para este proyecto nos hemos enfocado en hacer una pagina de venta de videojuegos aquí expondremos su funcionamiento y pruebas de las mismas.



Entidades y sus PKs:

plataformas → PK: id (Long, IDENTITY)

videojuegos → PK: id, FK: plataforma_id

clientes → PK: id

pedidos → PK: id, FK: cliente_id

pedidos_videojuegos → tabla intermedia con FK pedido_id + videojuego_id

2.2 Relaciones JPA:

Entidades	Tipo	FK / Tabla intermedia	Anotaciones JPA
Plataforma → Videojuego	@OneToMany	plataforma_id	@OneToMany(mappedBy) en Plataforma · @ManyToOne @JoinColumn en Videojuego · @JsonIgnore en el lado OneToMany
Cliente → Pedido	@OneToMany	cliente_id	@OneToMany(mappedBy, cascade=ALL) en Cliente · @ManyToOne @JoinColumn en Pedido · @JsonIgnore en el lado OneToMany
Pedido ↔ Videojuego	@ManyToMany	pedidos_videojuegos	@ManyToMany @JoinTable en Pedido (lado dueño) · @ManyToMany(mappedBy) @JsonIgnore en Videojuego

3. Lista de endpoints con método, ruta y descripción

Todos los endpoints tienen la base /api/v1/. Los endpoints de lectura (GET) son públicos. Los de escritura (POST, PUT, DELETE) requieren autenticación Basic Auth (usuario: admin / contraseña: admin123).

3.1 Plataformas

Método	Ruta	Descripción	Auth
GET	/api/v1/plataformas	Obtener todas las plataformas	No
GET	/api/v1/plataformas/{id}	Obtener plataforma por ID · 404 si no existe	No
POST	/api/v1/plataformas	Crear nueva plataforma · 201 Created	Sí
PUT	/api/v1/plataformas/{id}	Actualizar plataforma · 404 si no existe	Sí
DELETE	/api/v1/plataformas/{id}	Eliminar plataforma · 204 No Content / 404	Sí
GET	/api/v1/plataformas/buscar?nombre=&fabricante=	Buscar por nombre y/o fabricante (params opcionales)	No
GET	/api/v1/plataformas/nombre/{nombre}	Obtener plataforma por nombre exacto	No
GET	/api/v1/plataformas/{id}/contar-videojuegos	Contar videojuegos de una plataforma (@Query JPQL)	No

3.2 Videojuegos

Método	Ruta	Descripción	Auth
GET	/api/v1/videojuegos	Obtener todos los videojuegos	No
GET	/api/v1/videojuegos/{id}	Obtener videojuego por ID · 404 si no existe	No
POST	/api/v1/videojuegos	Crear nuevo videojuego · 201 Created	Sí
PUT	/api/v1/videojuegos/{id}	Actualizar videojuego · 404 si no existe	Sí
DELETE	/api/v1/videojuegos/{id}	Eliminar videojuego · 204 No Content / 404	Sí
GET	/api/v1/videojuegos/buscar?titulo=&sortBy=&order=	Buscar por título con ordenación dinámica	No

Método	Ruta	Descripción	Auth
GET	/api/v1/videojuegos/genero/{genero}	Filtrar por género	No
GET	/api/v1/videojuegos/plataforma/{nombre}	Filtrar por nombre de plataforma (relación)	No
GET	/api/v1/videojuegos/plataforma/id/{id}	Filtrar por ID de plataforma	No
GET	/api/v1/videojuegos/filtro?genero=&plataforma=	Filtro combinado género + plataforma	No
GET	/api/v1/videojuegos/precio-maximo?precio=	Videojuegos con precio $\leq$ valor dado	No
GET	/api/v1/videojuegos/en-stock	Videojuegos con stock > 0	No
GET	/api/v1/videojuegos/sin-pedidos	Videojuegos sin ningún pedido (@Query JPQL)	No
GET	/api/v1/videojuegos/mas-vendidos	Videojuegos ordenados por número de pedidos (@Query)	No
GET	/api/v1/videojuegos/{id}/contar-pedidos	Contar pedidos de un videojuego (@Query JPQL)	No

3.3 Clientes

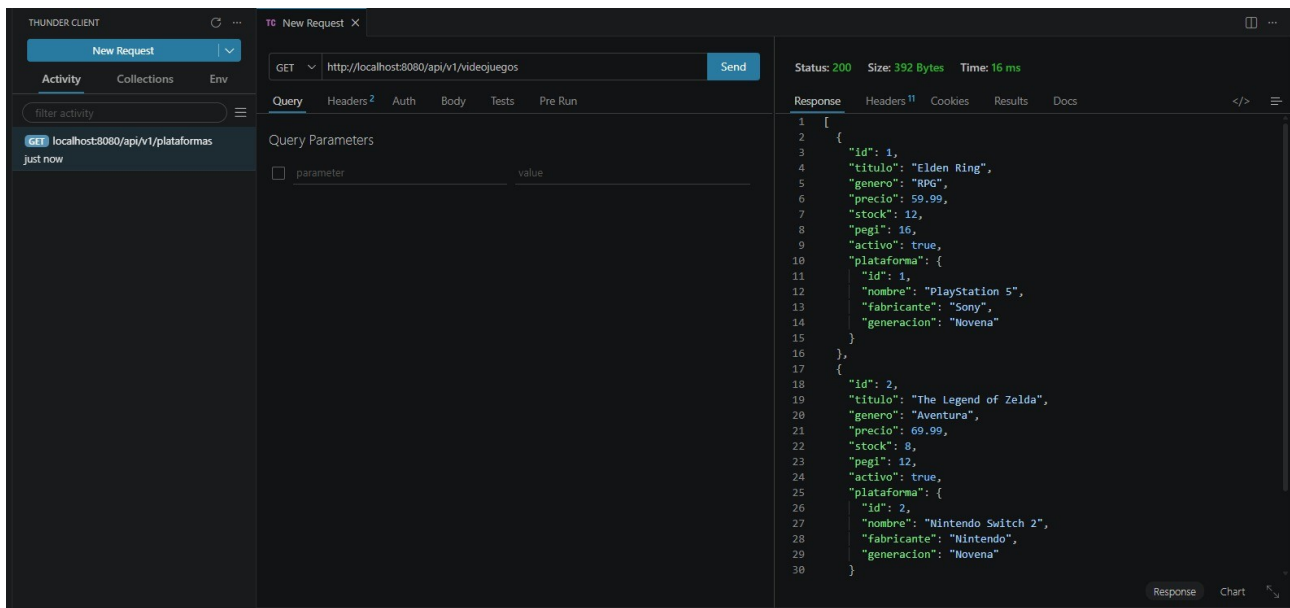
Método	Ruta	Descripción	Auth
GET	/api/v1/clientes	Obtener todos los clientes	No
GET	/api/v1/clientes/{id}	Obtener cliente por ID · 404 si no existe	No
POST	/api/v1/clientes	Crear nuevo cliente · 201 Created	Sí
PUT	/api/v1/clientes/{id}	Actualizar cliente · 404 si no existe	Sí
DELETE	/api/v1/clientes/{id}	Eliminar cliente · 204 No Content / 404	Sí
GET	/api/v1/clientes/buscar?nombre=	Buscar clientes por nombre (param opcional)	No
GET	/api/v1/clientes/email/{email}	Obtener cliente por email	No
GET	/api/v1/clientes/{id}/contar-pedidos	Contar pedidos de un cliente (@Query JPQL)	No
GET	/api/v1/clientes/con-minimo-pedidos?minimo=	Clientes con mínimo N pedidos (@Query HAVING)	No
GET	/api/v1/clientes/por-videojuego/{id}	Clientes que compraron un videojuego (@Query JOIN)	No

3.4 Pedidos

Método	Ruta	Descripción	Auth
GET	/api/v1/pedidos	Obtener todos los pedidos	No
GET	/api/v1/pedidos/{id}	Obtener pedido por ID · 404 si no existe	No
POST	/api/v1/pedidos	Crear nuevo pedido · 201 Created	Sí
PUT	/api/v1/pedidos/{id}	Actualizar pedido · 404 si no existe	Sí
DELETE	/api/v1/pedidos/{id}	Eliminar pedido · 204 No Content / 404	Sí
GET	/api/v1/pedidos/cliente/{clienteId}	Pedidos de un cliente concreto	No
GET	/api/v1/pedidos/estado/{estado}	Filtrar pedidos por estado	No
GET	/api/v1/pedidos/buscar?clienteId=&estado=&inicio=&fin=&sortBy=&order=	Búsqueda avanzada con múltiples filtros opcionales	No
GET	/api/v1/pedidos/videojuego/{id}	Pedidos que contienen un videojuego (@Query JOIN)	No
GET	/api/v1/pedidos/total-superior-promedio	Pedidos con total > media global (@Query subconsulta)	No
GET	/api/v1/pedidos/con-minimo-videojuegos?minimo=	Pedidos con mínimo N videojuegos (@Query HAVING)	No
GET	/api/v1/pedidos/cliente/{id}/resumen?inicio=&fin=	Resumen de pedidos y gasto de un cliente en periodo	No
POST	/api/v1/pedidos/{pedidoId}/videojuegos/{videojuegoId}	Añadir videojuego a un pedido (ManyToMany)	Sí
DELETE	/api/v1/pedidos/{pedidoId}/videojuegos/{videojuegoId}	Quitar videojuego de un pedido (ManyToMany)	Sí

4.Capturas o vídeo de las tablas generadas en la BD.

Get videojuegos



THUNDER CLIENT

New Request

Activity Collections Env

filter activity

GET localhost:8080/api/v1/plataformas just now

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

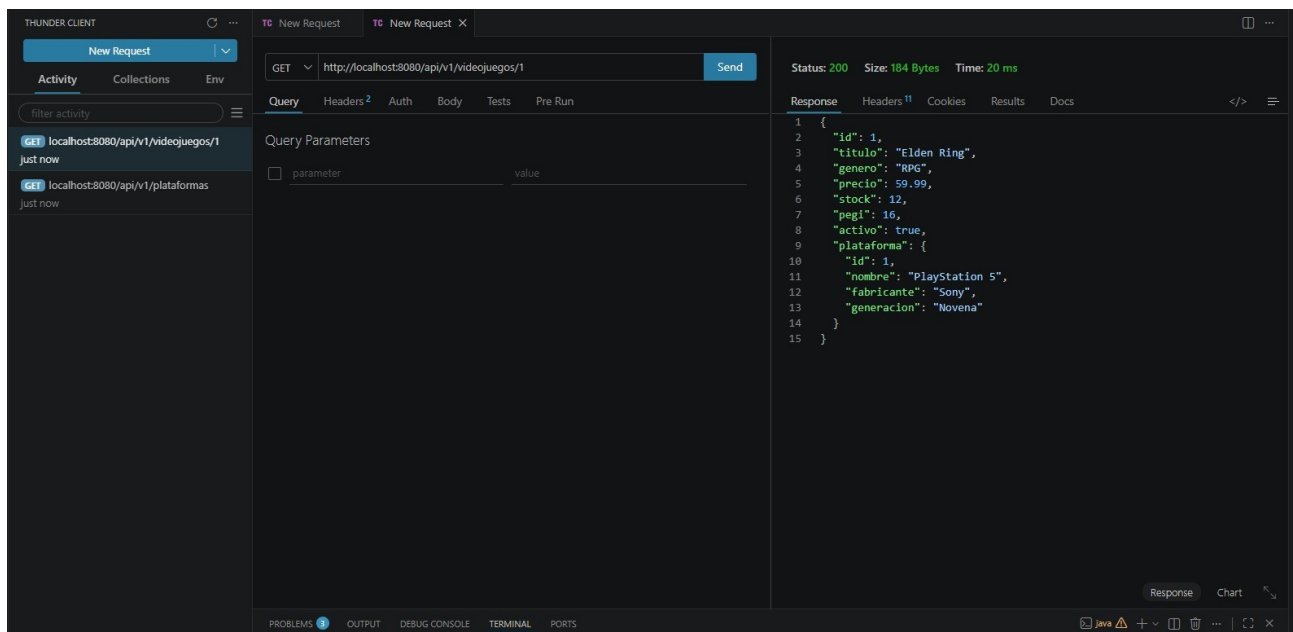
parameter value

Status: 200 Size: 392 Bytes Time: 16 ms

Response Headers 11 Cookies Results Docs

```
1 [
2   {
3     "id": 1,
4     "titulo": "Elden Ring",
5     "genero": "RPG",
6     "precio": 59.99,
7     "stock": 12,
8     "pegi": 16,
9     "activo": true,
10    "plataforma": {
11      "id": 1,
12      "nombre": "PlayStation 5",
13      "fabricante": "Sony",
14      "generacion": "Novena"
15    }
16  },
17  {
18    "id": 2,
19    "titulo": "The Legend of Zelda",
20    "genero": "Aventura",
21    "precio": 69.99,
22    "stock": 8,
23    "pegi": 12,
24    "activo": true,
25    "plataforma": {
26      "id": 2,
27      "nombre": "Nintendo Switch 2",
28      "fabricante": "Nintendo",
29      "generacion": "Novena"
30    }
31  }
32 ]
```

Get videojuegos por id 1



THUNDER CLIENT

New Request New Request

Activity Collections Env

filter activity

GET localhost:8080/api/v1/videojuegos/1 just now

GET localhost:8080/api/v1/plataformas just now

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

parameter value

Status: 200 Size: 184 Bytes Time: 20 ms

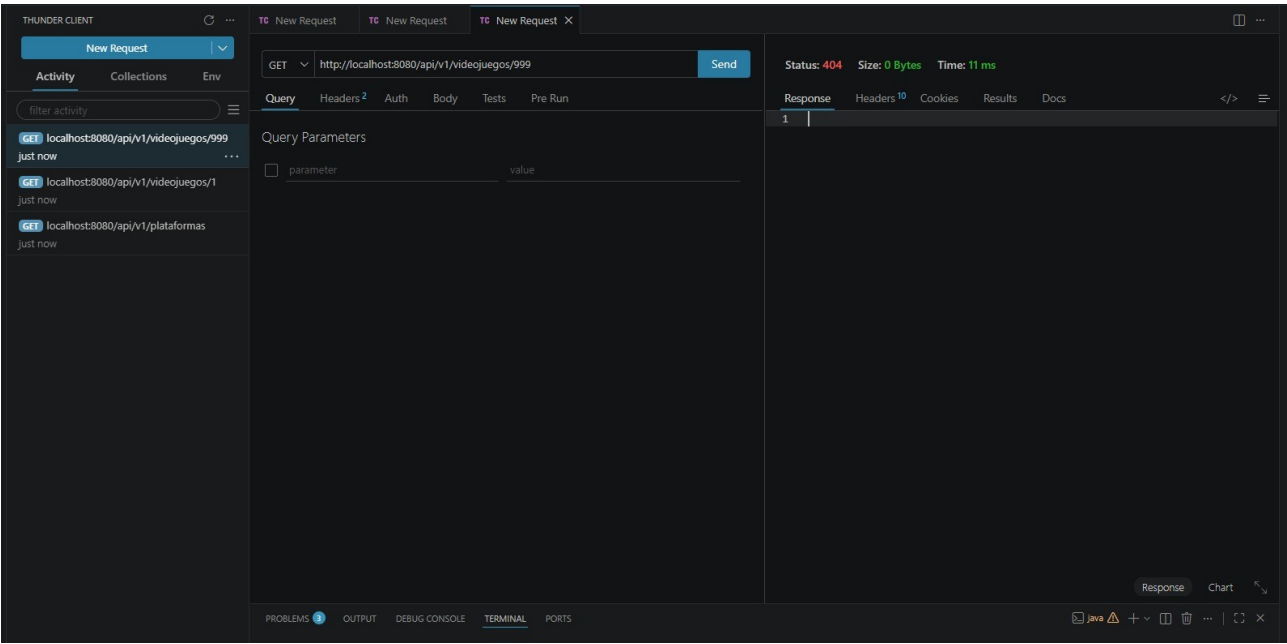
Response Headers 11 Cookies Results Docs

```
1 {
2   "id": 1,
3   "titulo": "Elden Ring",
4   "genero": "RPG",
5   "precio": 59.99,
6   "stock": 12,
7   "pegi": 16,
8   "activo": true,
9   "plataforma": {
10     "id": 1,
11     "nombre": "PlayStation 5",
12     "fabricante": "Sony",
13     "generacion": "Novena"
14   }
15 }
```

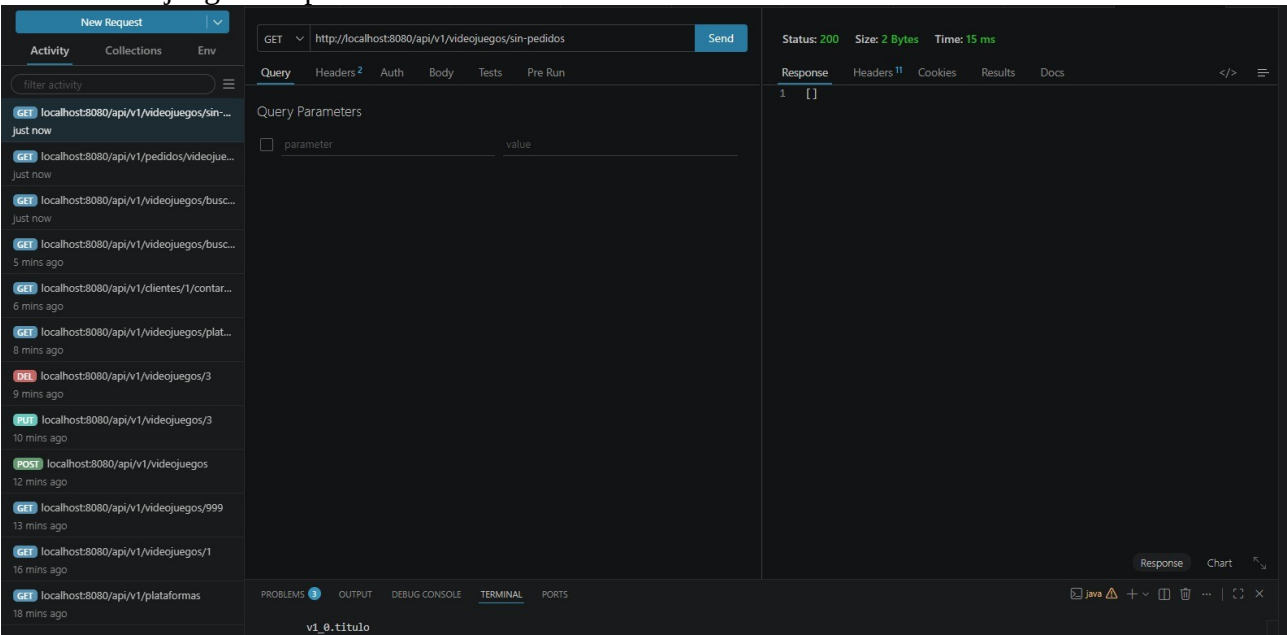
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

java

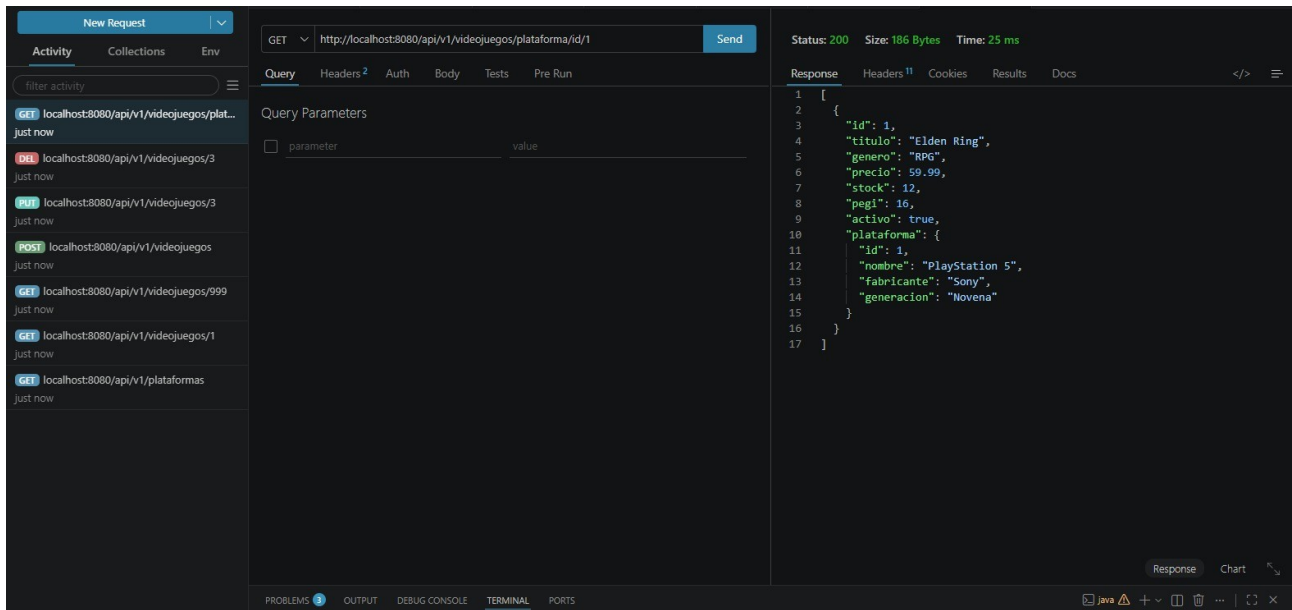
Get error 404



Get de videojuegos sin-pedidos



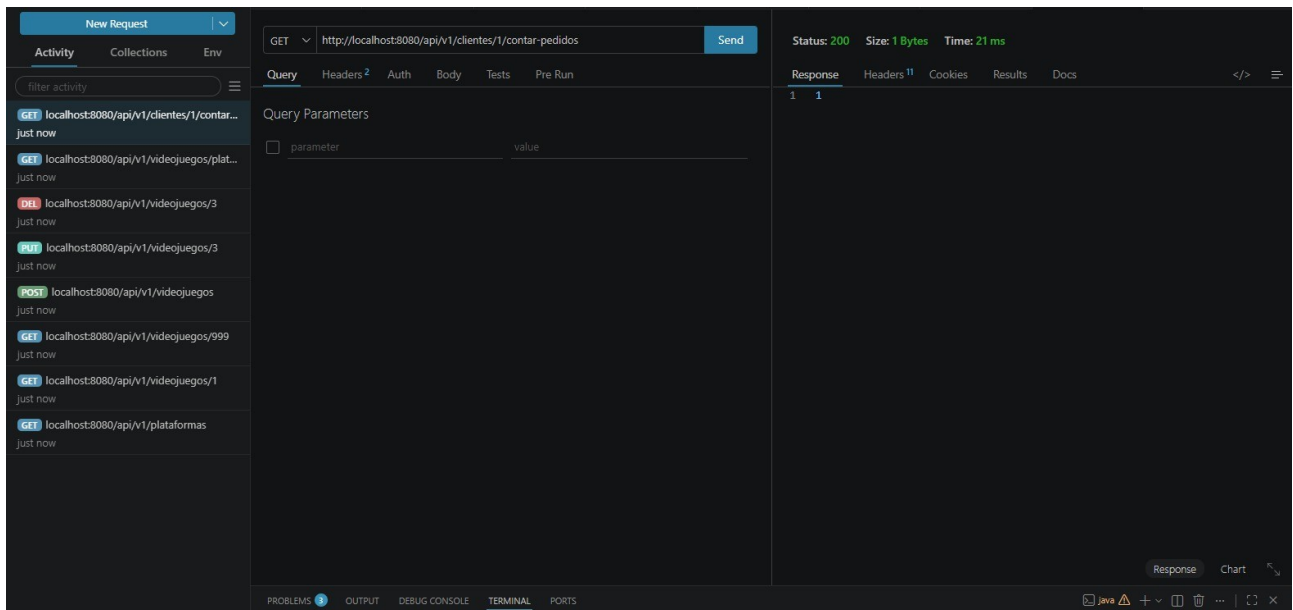
Get de ids de plataformas que empiezan por 1



The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/v1/videojuegos/plataforma/id/1`. The response is a JSON object with the following structure:

```
1 [
2   {
3     "id": 1,
4     "titulo": "Elden Ring",
5     "genero": "RPG",
6     "precio": 59.99,
7     "stock": 12,
8     "pEGI": 16,
9     "activo": true,
10    "plataforma": {
11      "id": 1,
12      "nombre": "PlayStation 5",
13      "fabricante": "Sony",
14      "generacion": "Novena"
15    }
16  }
17 ]
```

Get de clientes



The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/v1/clientes/1/contar-pedidos`. The response is a simple JSON object with the value 1:

```
1 1
```

Get titulo por nombre de titulo

The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/v1/videojuegos/buscar?titulo=ring`. The response is a JSON object with the following structure:

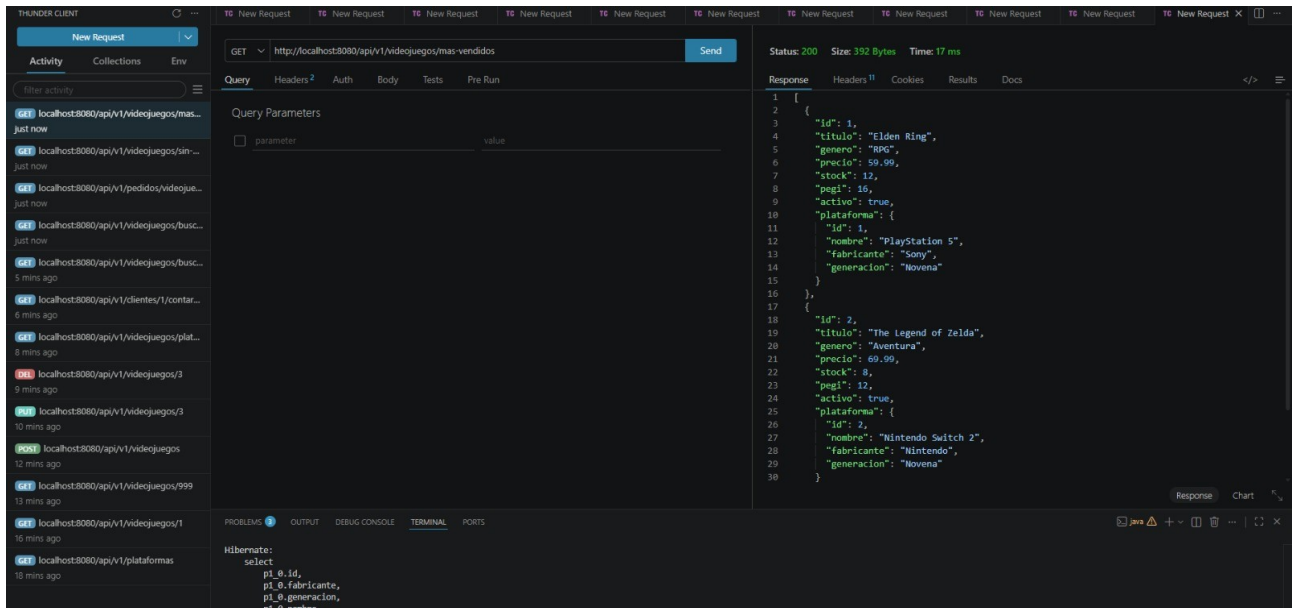
```
1 [
2   {
3     "id": 1,
4     "titulo": "Elden Ring",
5     "genero": "RPG",
6     "precio": 59.99,
7     "stock": 12,
8     "peli": 16,
9     "activo": true,
10    "plataforma": {
11      "id": 1,
12      "nombre": "PlayStation 5",
13      "fabricante": "Sony",
14      "generacion": "Novena"
15    }
16  }
17 ]
```

Get pedidos de videojuegos donde la id es 1

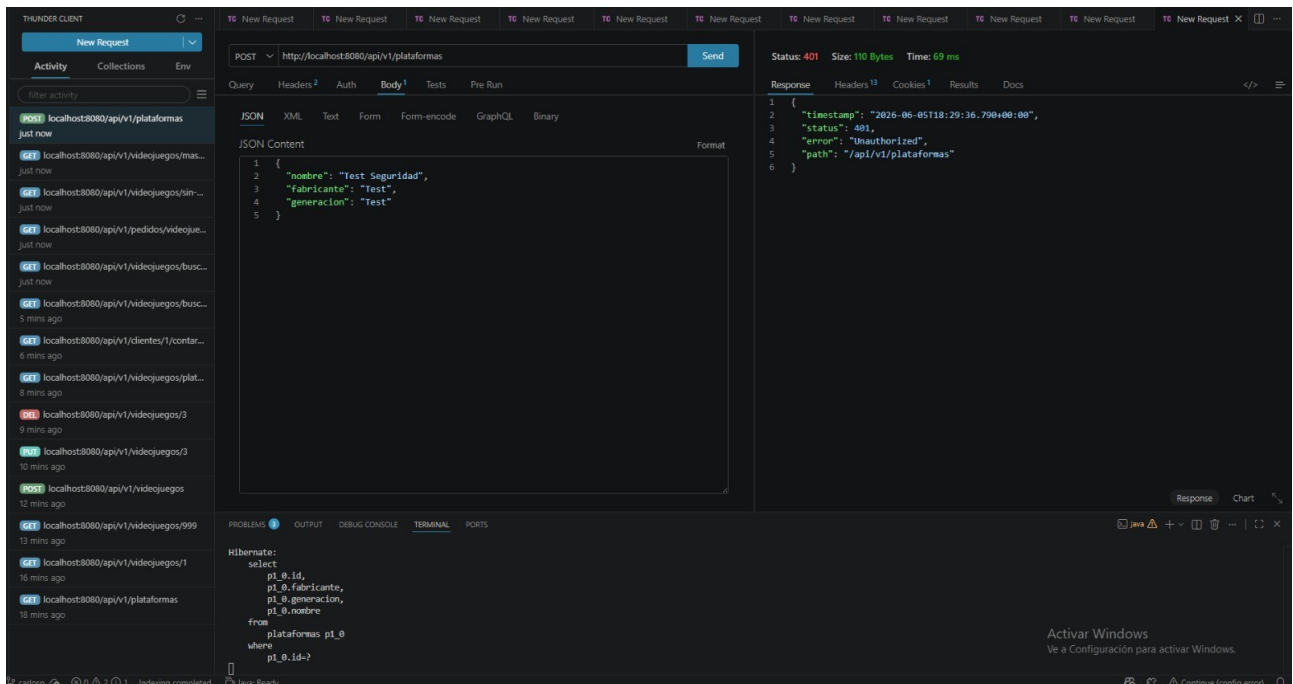
The screenshot shows the Postman interface with a GET request to `http://localhost:8080/api/v1/pedidos/videojuego/1`. The response is a JSON object with the following structure:

```
1 [
2   {
3     "id": 1,
4     "fecha": "2026-06-05",
5     "estado": "pagado",
6     "total": 129.98,
7     "observaciones": "Pedido inicial de prueba",
8     "cliente": {
9       "id": 1,
10      "nombre": "Carlos Perdomo",
11      "email": "carlos@example.com",
12      "telefono": "600111222",
13      "direccion": "Calle Principal 1"
14    },
15    "videojuegos": [
16      {
17        "id": 1,
18        "titulo": "Elden Ring",
19        "genero": "RPG",
20        "precio": 59.99,
21        "stock": 12,
22        "peli": 16,
23        "activo": true,
24        "plataforma": {
25          "id": 1,
26          "nombre": "PlayStation 5",
27          "fabricante": "Sony",
28          "generacion": "Novena"
29        }
30      }
31    ]
32  }
33 ]
```

Get videojuegos por mas-vendido



Post de plataforma nueva añadida dando error por no tener autorización



Get videojuego por el titulo “ring”, sort by “precio” y ordenado en descendiente

The screenshot shows the Thunder Client interface. On the left, the 'Activity' tab lists several requests. The main panel shows a GET request to `localhost:8080/api/v1/videojuegos/buscar?titulo=ring&sortBy=precio&order=desc`. The 'Query Parameters' section shows `titulo=ring`, `sortBy=precio`, and `order=desc`. The 'Response' tab shows a JSON array with one object:

```
1 [
2   {
3     "id": 1,
4     "titulo": "Elden Ring",
5     "genero": "RPG",
6     "precio": 59.99,
7     "stock": 12,
8     "pegil": 16,
9     "activo": true,
10    "plataforma": {
11      "id": 1,
12      "nombre": "PlayStation 5",
13      "fabricante": "Sony",
14      "generacion": "Novena"
15    }
16  }
17 ]
```

Post de nueva plataforma a\u00f1adida

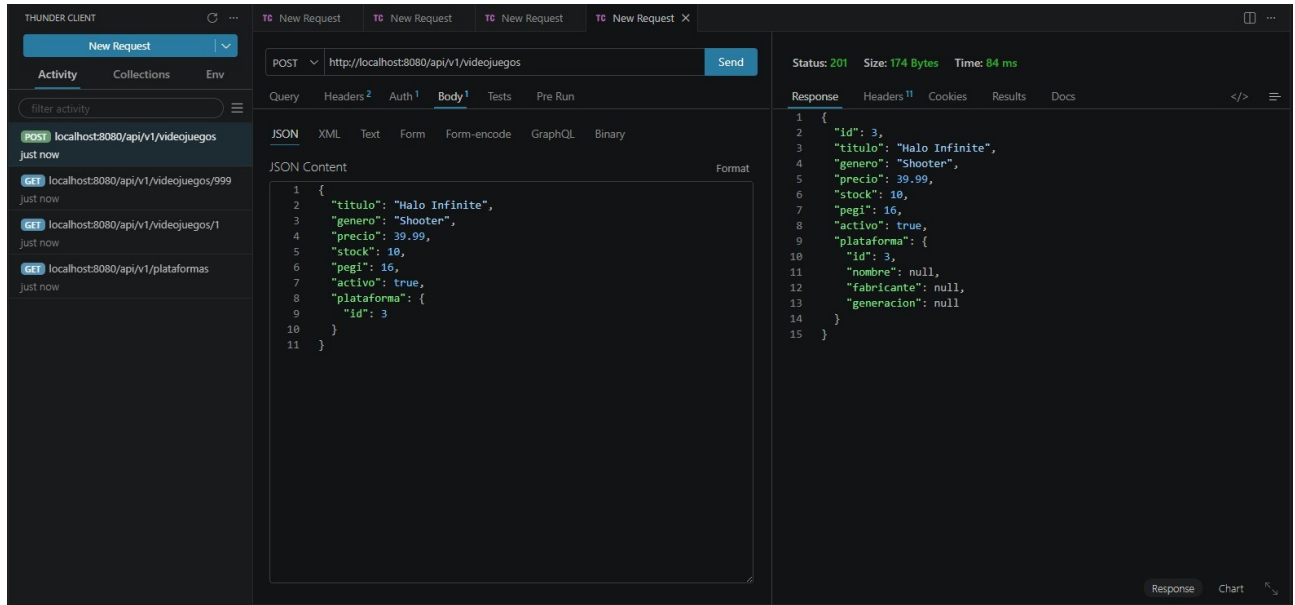
The screenshot shows the Thunder Client interface. On the left, the 'Activity' tab lists several requests. The main panel shows a POST request to `http://localhost:8080/api/v1/plataformas`. The 'Body' tab shows a JSON object:

```
1 {
2   "nombre": "Xbox Series X",
3   "fabricante": "Microsoft",
4   "generacion": "Novena"
5 }
```

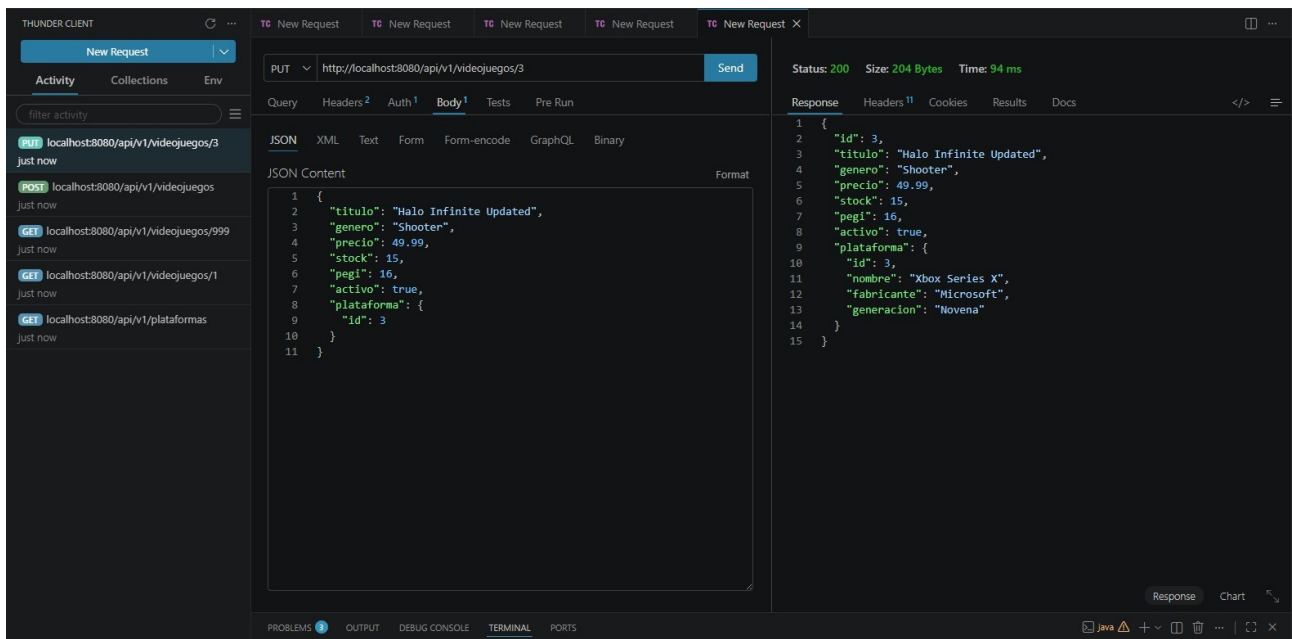
The 'Response' tab shows a JSON object:

```
1 {
2   "id": 3,
3   "nombre": "Xbox Series X",
4   "fabricante": "Microsoft",
5   "generacion": "Novena"
6 }
```

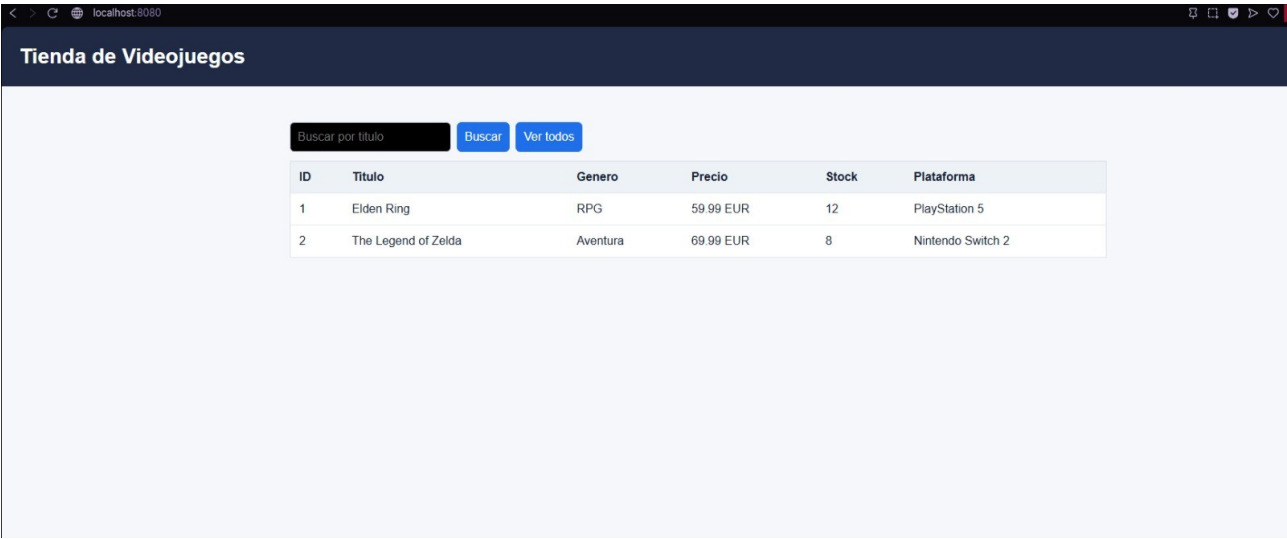
Post añadido nuevo juego a “videojuegos”



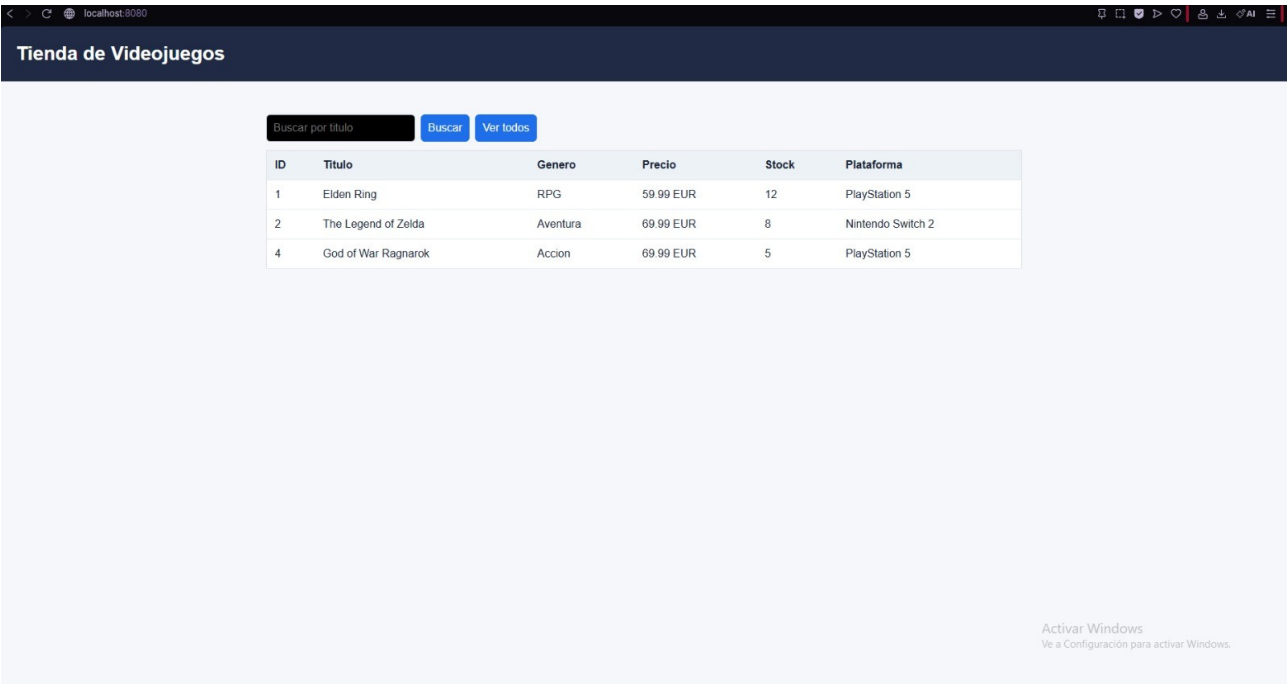
Put de actualización del juego “Halo infinite”



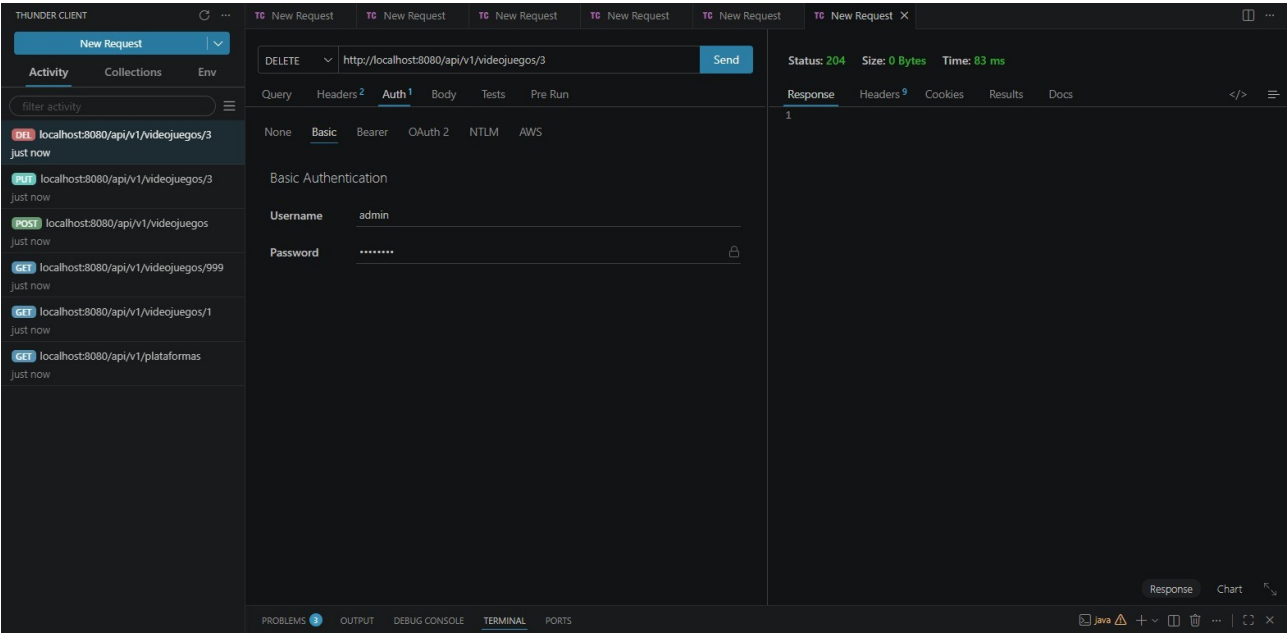
Visualización de la app de la tienda



Actualización de la app



Del con autenticación para eliminar el videojuego 3



4. Decisiones técnicas justificadas

4.1 Base de datos: H2 en memoria

Se eligió H2 en memoria por su comodidad en desarrollo: no requiere instalar ningún servidor externo, la base de datos se genera automáticamente al arrancar la aplicación con `spring.jpa.hibernate.ddl-auto=create`, y la consola web de H2 permite inspeccionar las tablas en tiempo real en `http://localhost:8080/h2-console`. Para producción, bastará cambiar el `datasource` a MySQL sin tocar el código JPA.

4.2 Arquitectura por capas: Controller → Service → Repository

Se respeta estrictamente la separación en tres capas. El Controller nunca accede directamente al Repository; siempre pasa por el Service. Esta decisión facilita el testing unitario (se puede mockear el Service), mantiene el principio de responsabilidad única y permite cambiar la lógica de negocio sin modificar la capa de presentación.

4.3 Optional y ResponseEntity

Todos los `findById` y búsquedas por campo único devuelven `Optional<T>`. En el Controller se usa el patrón `.map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build())` para evitar llamar a `.get()` sin comprobar si hay valor, lo que lanzaría `NoSuchElementException`. Esto garantiza respuestas HTTP semánticamente correctas (200 vs 404) sin excepciones en tiempo de ejecución.

4.4 @JsonIgnore para evitar referencias circulares

En las relaciones bidireccionales, el lado inverso (el que tiene `mappedBy`) lleva `@JsonIgnore` para que Jackson no serialice la lista completa al devolver el objeto, evitando así bucles infinitos en la serialización JSON. Por ejemplo, Plataforma tiene `@JsonIgnore` en su lista de Videojuegos, y Videojuego tiene `@JsonIgnore` en su Set de Pedidos.

4.5 @ManyToMany con @JoinTable explícita

La relación ManyToMany entre Pedido y Videojuego se declara con `@JoinTable` en la entidad Pedido (lado dueño), nombrando explícitamente la tabla intermedia `pedidos_videojuegos` y sus columnas `pedido_id` y `videojuego_id`. Esto da control total sobre el nombre de la tabla y las FK, en lugar de depender del nombre generado automáticamente por Hibernate.

4.6 Métodos derivados y @Query JPQL

Se usan métodos derivados de Spring Data JPA (`findByNombreContainingIgnoreCase`, `findByPrecioLessThanEqual`, etc.) cuando la consulta es simple y expresable con el DSL de nombres. Se recurre a `@Query JPQL` cuando la consulta no se puede expresar limpiamente con un método derivado: subconsultas (total superior al promedio), `HAVING` con `COUNT`, navegación a través de varias relaciones, o proyecciones especiales. La diferencia clave entre JPQL y SQL es que JPQL opera sobre entidades y sus propiedades Java, no sobre tablas y columnas de la base de datos.

4.7 Seguridad con Spring Security Basic Auth

Se implementa Spring Security con autenticación HTTP Basic para proteger los endpoints de escritura. Los GET son públicos (permitAll). Los POST, PUT, PATCH y DELETE requieren autenticación. Se define un usuario en memoria (admin/admin123) con rol ADMIN. La consola H2 también es accesible sin autenticación con CSRF desactivado y frameOptions configurado para sameOrigin, necesario para que el iframe de la consola H2 funcione correctamente.

4.8 Validación con @Valid y manejo global de excepciones

Las entidades llevan anotaciones de validación Bean Validation: @NotBlank, @NotNull, @Size, @Email, @PositiveOrZero. Al usar @Valid en los @RequestBody de los controllers, Spring valida automáticamente los datos de entrada y lanza MethodArgumentNotValidException si no son válidos.

El GlobalExceptionHandler con @ControllerAdvice intercepta estas excepciones y devuelve respuestas JSON estructuradas con timestamp, mensaje de error y detalle de los campos inválidos, en lugar del error HTML genérico de Spring Boot. También gestiona DataIntegrityViolationException (datos duplicados) e IllegalArgumentException.

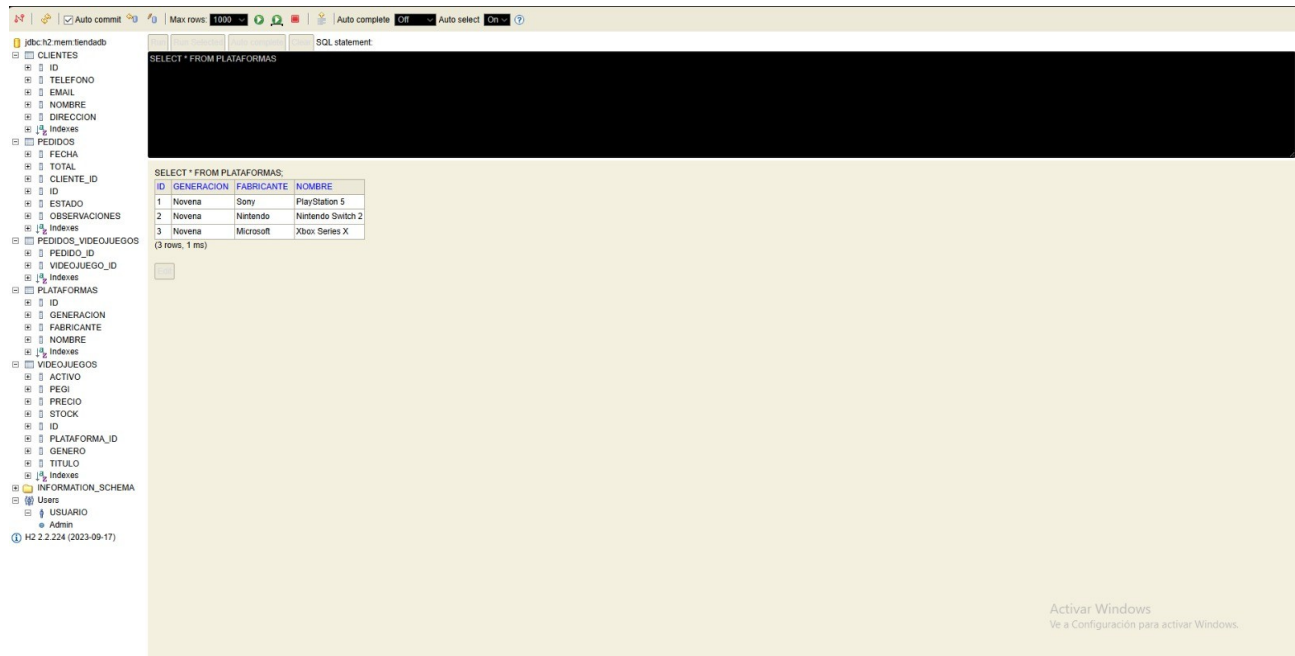
4.9 Sort dinámico en búsquedas

El endpoint de búsqueda de videojuegos acepta los parámetros sortBy y order para permitir ordenación dinámica sin escribir múltiples métodos. Se construye un objeto Sort de Spring Data en la capa Service y se pasa al repositorio, que acepta Sort como parámetro adicional en findByTituloContainingIgnoreCase.

5. Capturas de las tablas en la base de datos

Las siguientes capturas muestran las tablas generadas por JPA/Hibernate al arrancar la aplicación, visualizadas a través de la consola H2 (http://localhost:8080/h2-console) o con datos de prueba cargados por el DataInitializer.

Plataformas

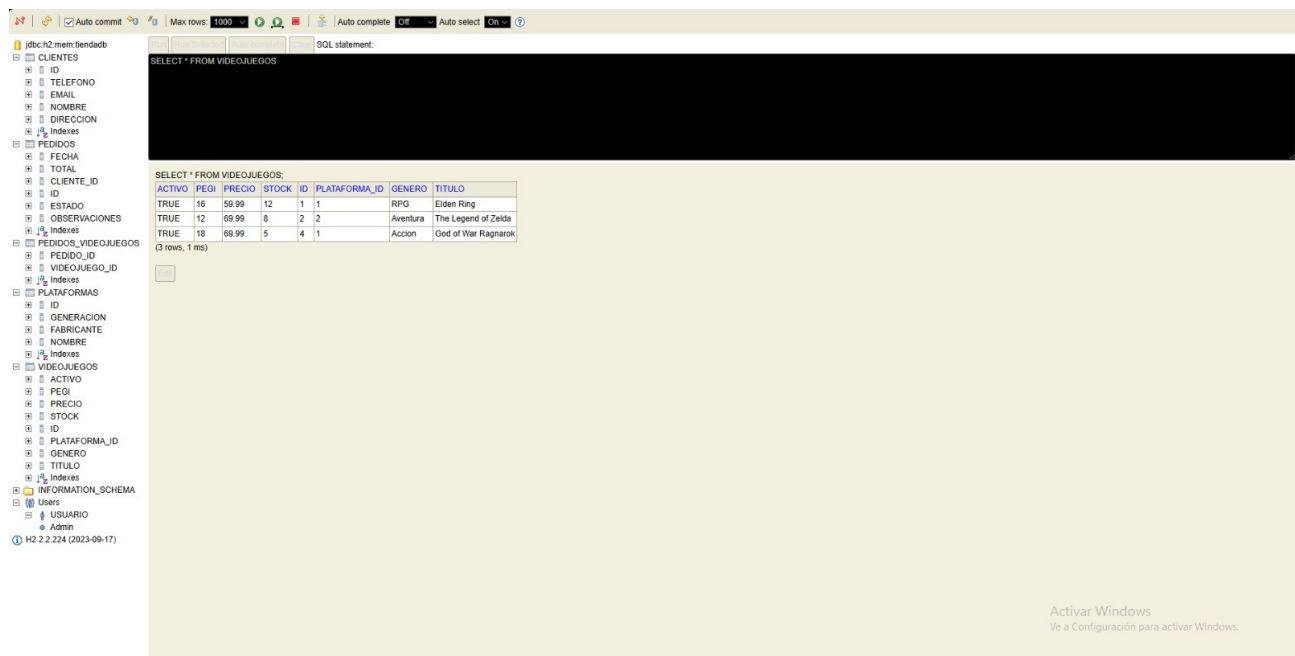


The screenshot shows the H2 console interface. On the left, a tree view displays the database schema, including tables like CLIENTES, PEDIDOS, PEDIDOS_VIDEOJUEGOS, and PLATAFORMAS. The 'PLATAFORMAS' table is selected. The main area displays the SQL statement 'SELECT * FROM PLATAFORMAS;' and the resulting table data:

ID	GENERACION	FABRICANTE	NOMBRE
1	Novena	Sony	PlayStation 5
2	Novena	Nintendo	Nintendo Switch 2
3	Novena	Microsoft	Xbox Series X

Below the table, it indicates '(3 rows, 1 ms)'. The bottom right corner of the console shows the text 'Activar Windows' and 'Ve a Configuración para activar Windows.'

Videojuegos



The screenshot shows the H2 console interface. On the left, the same database schema tree is visible. The 'VIDEOJUEGOS' table is selected. The main area displays the SQL statement 'SELECT * FROM VIDEOJUEGOS;' and the resulting table data:

ACTIVO	PEGI	PRECIO	STOCK	ID	PLATAFORMA_ID	GENERO	TITULO
TRUE	16	59.99	12	1	1	RPG	Elden Ring
TRUE	12	69.99	8	2	2	Aventura	The Legend of Zelda
TRUE	18	69.99	5	4	1	Action	God of War Ragnarok

Below the table, it indicates '(3 rows, 1 ms)'. The bottom right corner of the console shows the text 'Activar Windows' and 'Ve a Configuración para activar Windows.'

Cientes

- jdbc:h2:mem:tiendadb
 - CLIENTES
 - ID
 - TELEFONO
 - EMAIL
 - NOMBRE
 - DIRECCION
 - Indexes
 - PEDIDOS
 - FECHA
 - TOTAL
 - CLIENTE_ID
 - ID
 - ESTADO
 - OBSERVACIONES
 - Indexes
 - PEDIDOS_VIDEOJUEGOS
 - PEDIDO_ID
 - VIDEOJUEGO_ID
 - Indexes
 - PLATAFORMAS
 - ID
 - GENERACION
 - FABRICANTE
 - NOMBRE
 - Indexes
 - VIDEOJUEGOS
 - ACTIVO
 - PEGI
 - PRECIO
 - STOCK
 - ID
 - PLATAFORMA_ID
 - GENERO
 - TITULO
 - Indexes
 - INFORMATION_SCHEMA
 - Users
 - USUARIO
 - Admin
- H2 2.2.224 (2023-09-17)

SQL statement:

SELECT * FROM CLIENTES;

SELECT * FROM CLIENTES;

ID	TELEFONO	EMAIL	NOMBRE	DIRECCION
1	600111222	carlos@example.com	Carlos Pardo	Calle Principal 1

(1 row, 2 ms)

Activar Windows

Ve a Configuración para activar Windows.

Pedidos

- jdbc:h2:mem:tiendadb
 - CLIENTES
 - ID
 - TELEFONO
 - EMAIL
 - NOMBRE
 - DIRECCION
 - Indexes
 - PEDIDOS
 - FECHA
 - TOTAL
 - CLIENTE_ID
 - ID
 - ESTADO
 - OBSERVACIONES
 - Indexes
 - PEDIDOS_VIDEOJUEGOS
 - PEDIDO_ID
 - VIDEOJUEGO_ID
 - Indexes
 - PLATAFORMAS
 - ID
 - GENERACION
 - FABRICANTE
 - NOMBRE
 - Indexes
 - VIDEOJUEGOS
 - ACTIVO
 - PEGI
 - PRECIO
 - STOCK
 - ID
 - PLATAFORMA_ID
 - GENERO
 - TITULO
 - Indexes
 - INFORMATION_SCHEMA
 - Users
 - USUARIO
 - Admin
- H2 2.2.224 (2023-09-17)

SQL statement:

SELECT * FROM PEDIDOS;

SELECT * FROM PEDIDOS;

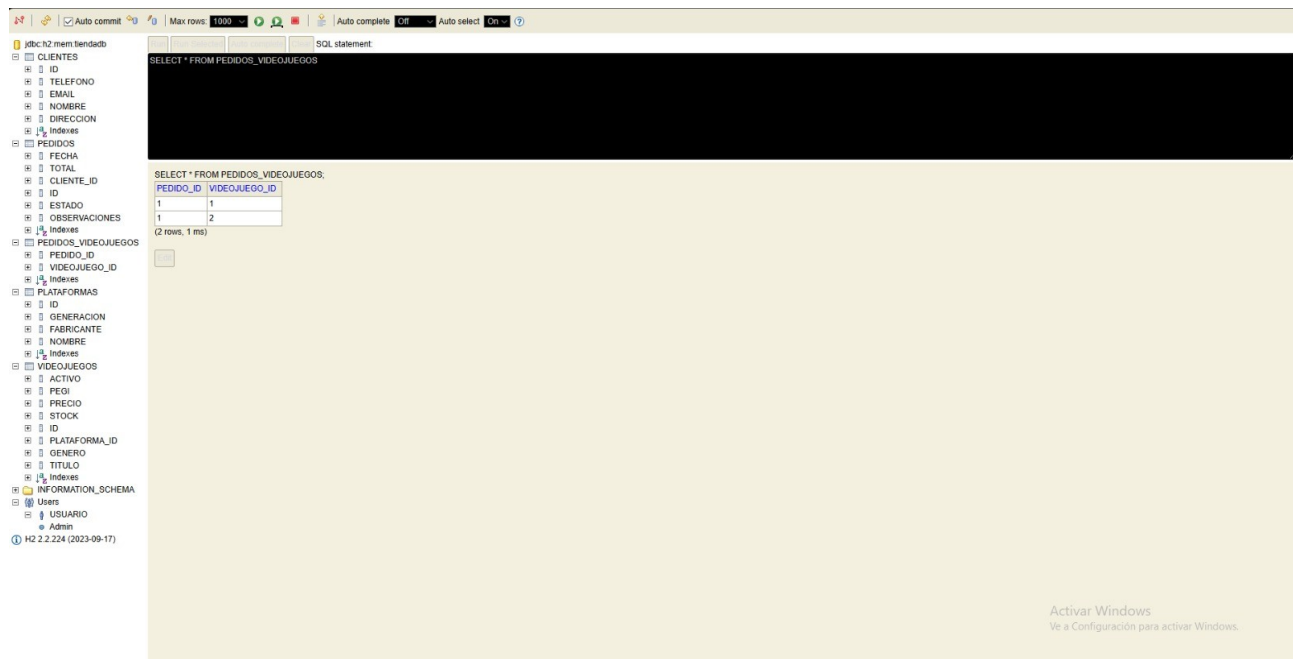
FECHA	TOTAL	CLIENTE_ID	ID	ESTADO	OBSERVACIONES
2025-06-05	129.98	1	1	pagado	Pedido inicial de prueba

(1 row, 0 ms)

Activar Windows

Ve a Configuración para activar Windows.

PEDIDOS_VIDEOJUEGOS



6. Instrucciones para arrancar el proyecto

Requisitos previos

- Java 17 o superior
- Maven (o usar el wrapper incluido `./mvnw`)
- No se necesita base de datos externa (H2 en memoria)

Pasos

- Clonar el repositorio: `git clone https://github.com/perdomomoralescarlos-ctrl/Ut6Pro.git`
- Entrar en la carpeta: `cd Ut6Pro`
- Arrancar: `./mvnw spring-boot:run`
- La API estará disponible en: `http://localhost:8080/api/v1/`
- Consola H2: `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:tiendadb`)

Credenciales para endpoints de escritura

- Usuario: admin
- Contraseña: admin123
- Método: HTTP Basic Auth (configurar en Postman: Authorization > Basic Auth)